

GhostMind — Documento de Concepção (Pitch — Etapa 1)

Projeto N3 — Sistema Inteligente com Agentes Disciplina: Inteligência Artificial — Engenharia de Software Integrantes: [preencher: 3 a 4 nomes] Data: 09/06/2026

| Sugestão de nome — "GhostMind". Troquem à vontade (ex.: "Caça no Labirinto", "PacHunt").

1. Release Notes — v0.1 (conceitual)

Nome e versão

GhostMind v0.1 — Simulador de perseguição e fuga com agentes inteligentes em labirinto (inspirado no Pac-Man).

Resumo executivo

GhostMind é um simulador interativo, inspirado no Pac-Man, em que múltiplos agentes-fantasma autônomos caçam um agente Pac-Man num labirinto em grade. Cada fantasma planeja seu trajeto em tempo real com busca A*, enquanto o Pac-Man — autônomo ou controlado pelo jogador — decide suas fugas com busca gulosa (**Greedy Best-First**). O sistema resolve o problema de navegação e perseguição inteligente em ambiente dinâmico e demonstra, ao vivo, o comportamento e o desempenho de diferentes estratégias de busca informada.

Principais funcionalidades

1. **Labirinto jogável** com Pac-Man, pílulas, pílulas de poder e múltiplos fantasmas, em **modo jogador** (controle manual) e **modo IA** (Pac-Man autônomo).
2. **Fantasmas como agentes baseados em objetivos** que perseguem o Pac-Man recalculando, a cada passo, o caminho de menor custo com A* (heurística de distância de Manhattan) — replanejamento contínuo em ambiente que muda enquanto o agente delibera.
3. **Personalidades de fantasmas**: perseguidor direto (mira a posição atual), emboscador (mira a posição futura prevista) e cauteloso (recua ao se aproximar) — todos sobre o **mesmo motor A***, apenas com alvos diferentes.
4. **Inversão de comportamento por evento**: ao coletar uma pílula de poder, os fantasmas alternam de "caçar" para "fugir" do Pac-Man, evidenciando agentes que percebem e reagem a mudanças de estado do ambiente.
5. **Pac-Man autônomo** que coleta pílulas e foge usando **Greedy Best-First**, permitindo **comparar A* × Greedy ao vivo**.
6. **Painel de métricas em tempo real** (nós expandidos, comprimento do caminho e tempo de cálculo por agente) para evidenciar o impacto das heurísticas.

Tecnologias e técnicas de IA

- **Stack**: Python 3 com Pygame (loop e renderização do jogo) — facilmente adaptável a outra linguagem/engine.

- **Tipos de agente:** agentes baseados em objetivos (goal-based) com modelo interno do ambiente. O ambiente é **totalmente observável, multiagente, dinâmico, discreto e sequencial**, e essencialmente determinístico.
- **Heurísticas de busca:** A* (fantasmas) e **Greedy Best-First** (Pac-Man), ambas usando a **distância de Manhattan** — admissível e consistente em grid com movimento em 4 direções.
- **(Opcional) Naive Bayes:** classificação probabilística do nível de ameaça do estado de jogo, para ajustar a postura do Pac-Man autônomo. *Ver seção "Possíveis evoluções".*

Limitações conhecidas / fora de escopo (v0.1)

- **Labirinto estático e pré-definido** (sem editor de mapas nem geração procedural).
- **Sem aprendizado entre partidas:** a inteligência vem do replanejamento de busca a cada passo, não de treinamento.
- **Single-player local** apenas (sem rede ou multiplayer).
- Pontuação, vidas e progressão de fases entram de forma **simplificada**; o foco da v0.1 é o comportamento dos agentes.
- A camada **Naive Bayes é opcional** nesta versão (núcleo prioriza A* + Greedy).

Possíveis evoluções (v0.2+)

- **Naive Bayes** para classificar o nível de ameaça do estado (seguro / risco / crítico) e ajustar a estratégia do Pac-Man autônomo entre coletar e fugir, treinado com partidas simuladas.
- **Q-Learning** para o Pac-Man aprender políticas de fuga e coleta com a experiência.
- **Editor de labirintos** e geração procedural de mapas.
- **Modo competitivo entre IAs** (múltiplos Pac-Man ou estratégias rivais).
- **Mais heurísticas no comparador** (distância euclidiana, BFS/Dijkstra) no mesmo painel de métricas.

2. FAQ — Perguntas Frequentes

1. Como cada fantasma decide para onde se mover? A cada passo, o fantasma trata sua célula atual como nó inicial e a célula-alvo da sua personalidade (em geral a posição do Pac-Man) como objetivo, roda A* sobre o grafo do labirinto e executa apenas o **primeiro passo** do caminho retornado. Como o alvo se move, o caminho é recalculado continuamente — é esse replanejamento que faz o agente reagir ao ambiente dinâmico.

2. Quais são as entradas e saídas do sistema? **Entradas:** o mapa do labirinto (matriz de paredes/caminhos), as posições de Pac-Man e fantasmas, a localização das pílulas e, no modo jogador, as teclas de direção. **Saídas:** o próximo movimento de cada agente, o estado do jogo atualizado e renderizado na tela e as métricas de busca (nós expandidos, custo do caminho e tempo de cálculo).

3. Por que **A*** para os fantasmas e **Greedy** para o Pac-Man? E por que **distância de Manhattan**?

A* garante o caminho de menor custo, o que é ideal para o perseguidor que quer alcançar o Pac-Man de forma ótima. A **distância de Manhattan** é admissível e consistente num grid de 4 conexões (nunca superestima o custo real), preservando a otimalidade do **A*** com bom desempenho. O Pac-Man usa **Greedy Best-First** porque, ao fugir, decisões rápidas e "boas o suficiente" importam mais que a rota ótima: Greedy expande menos nós e responde mais rápido, ao custo de às vezes cair em becos — o que torna a comparação entre os dois didática e visível no painel de métricas.

4. O sistema aprende ao longo do tempo? Na **v0.1**, **não**: não há aprendizado entre partidas. Toda a inteligência dos agentes vem do replanejamento de busca a cada passo, não de treinamento. O aprendizado está previsto como evolução (**v0.2**): primeiro um **Naive Bayes** para avaliar ameaça e, mais adiante, **Q-Learning** para o Pac-Man aprender políticas de fuga e coleta com a experiência.

5. Onde entraria o classificador **Naive Bayes**? Como **camada opcional/futura**, o Naive Bayes classificaria o estado do Pac-Man em níveis de ameaça (seguro / risco / crítico) a partir de atributos como distância ao fantasma mais próximo, número de fantasmas em rota de colisão, pílulas de poder disponíveis e pílulas restantes. O modelo seria treinado com estados rotulados de partidas simuladas, e seu resultado ajustaria a postura do Pac-Man autônomo — priorizar coleta quando "seguro", priorizar fuga quando "crítico". É exatamente uma decisão **sob incerteza**, que é o caso de uso natural do classificador.

6. Como os fantasmas têm comportamentos diferentes se usam o mesmo algoritmo? Todos compartilham o **mesmo motor A***, mas recebem **alvos diferentes**: o perseguidor mira a célula atual do Pac-Man; o emboscador mira algumas células à frente da direção dele (predição simples); o cauteloso persegue, mas troca o alvo para um ponto de recuo quando a distância fica pequena. Isso gera personalidades distintas com custo mínimo de código.

7. O que acontece quando o Pac-Man pega uma pílula de poder? O estado global muda para "modo poder" por tempo limitado: os fantasmas **invertem o objetivo** e passam a maximizar a distância ao Pac-Man (fuga, via Greedy), enquanto o Pac-Man pode capturá-los. Isso demonstra agentes que percebem e reagem a mudanças no ambiente, alternando estratégia conforme o estado.

8. O ambiente é determinístico? Como garantir desempenho em tempo real? O ambiente é **discreto, totalmente observável e essencialmente determinístico** — cada ação leva a um estado previsível. Como o labirinto é pequeno (dezenas a poucas centenas de células), uma busca **A*** por agente a cada passo é barata; ainda assim, o replanejamento pode ser feito a cada **N** passos, e o painel de métricas comprova que o tempo de cálculo permanece na casa de milissegundos.